

WEBX - EXPERIMENT NO. 4

Name of Student	<u>Anushka Shahane</u>
Class Roll No	D15A / 54
D.O.P.	27/02/25
D.O.S.	6/03/25
Sign and Grade	

AIM : To design a Flask application that showcases URL building and demonstrates the use of HTTP methods (GET and POST) for handling user input and processing data.

PROBLEM STATEMENT :

Create a Flask application with the following requirements:

1. A homepage (/) with links to a "Profile" page and a "Submit" page using the `url_for()` function.
2. The "Profile" page (/profile/<username>) dynamically displays a user's name passed in the URL.
3. A "Submit" page (/submit) displays a form to collect the user's name and age. The form uses the POST method to send the data, and the server displays a confirmation message with the input.

Theory:

1. What is a route in Flask, and how is it defined?

A **route** in Flask is a URL pattern that is associated with a specific function, often referred to as a **view function**. When a user accesses a URL in a Flask application, Flask maps that URL to a Python function defined by the route. The purpose of a route is to handle incoming requests and return a response, such as an HTML page, JSON data, or other content.

In Flask, routes are defined using the `@app.route()` decorator, which binds a URL to a Python function.

Example:

```
@app.route('/')
```

```
def home():
```

```
return 'Welcome to the Homepage'
```

In this example, the `@app.route('/')` decorator specifies that the `home()` function will be executed when a user accesses the root URL (`/`) of the web application.

2. How can you pass parameters in a URL route?

In Flask, you can pass parameters in a URL route by using **dynamic route variables**. These variables are defined within the angle brackets (`<>`) in the route definition. When a request is made to that route, Flask will capture the value in the URL and pass it to the associated function as an argument.

Example:

```
@app.route('/greet/<name>')
```

```
def greet(name):
```

```
    return f'Hello, {name}!'
```

In this example, the route `/greet/<name>` will capture whatever string appears in place of `<name>` in the URL and pass it to the `greet()` function. So, a URL like `/greet/John` would return the response `Hello, John!`.

3. What happens if two routes in a Flask application have the same URL pattern?

If two routes in Flask have the same URL pattern, Flask will raise a `ValueError` during application startup. This is because URL patterns must be unique within the application to avoid ambiguity. When the application is loaded, Flask will not know which function to call when the same URL is accessed, and this will result in an error.

Example of duplicate routes:

```
@app.route('/home')
```

```
def home1():
```

```
    return 'This is the first home page.'
```

```
@app.route('/home')
```

```
def home2():
```

```
    return 'This is the second home page.'
```

In the above example, Flask will raise an error because both routes have the same URL (/home), so it can't determine which function should handle the request.

4. What are the commonly used HTTP methods in web applications?

HTTP methods define the action that is performed on the resource identified by a URL. The most commonly used HTTP methods in web applications are:

- **GET**: Used to retrieve data from the server. It's the default method used in most requests when accessing a web page.
- **POST**: Used to send data to the server, often used when submitting a form or creating a resource.
- **PUT**: Used to update an existing resource or replace a resource.
- **DELETE**: Used to delete a resource.
- **PATCH**: Used to update a resource, typically with partial data (similar to PUT but more specific).
- **OPTIONS**: Used to get the allowed methods for a specific URL or resource.

In Flask, you can specify which HTTP methods a route should handle by using the methods parameter in the `@app.route()` decorator.

Example:

```
@app.route('/submit', methods=['POST'])
```

```
def submit_data():
```

```
    return 'Data submitted successfully!'
```

In this example, the route /submit will only respond to POST requests.

5. What is a dynamic route in Flask?

A **dynamic route** in Flask refers to a route where part of the URL is variable and can change based on the request. This is achieved by defining route parameters in the URL pattern. These parameters can be accessed inside the view function as function arguments.

Dynamic routes allow you to create flexible URL patterns that can capture data from the URL itself.

Example:

```
@app.route('/user/<username>')
```

```
def show_user(username):
```

```
    return f'Hello, {username}!'
```

In this example, the route `/user/<username>` is dynamic because the `<username>` part of the URL is variable. Flask will capture the username from the URL and pass it to the `show_user()` function.

6. Write an example of a dynamic route that accepts a username as a parameter.

Here is an example of a dynamic route in Flask that accepts a username as a parameter in the URL:

```
@app.route('/user/<username>')

def user_profile(username):

    return f'Welcome to the profile of {username}'
```

If a user accesses the URL `/user/john`, the output would be `Welcome to the profile of john`.

You can also specify types for route parameters, such as integers or strings, to enforce constraints.

Example with a type parameter:

```
@app.route('/user/<int:id>')

def user_by_id(id):

    return f'User ID is {id}'
```

In this case, the route will only match if the `id` is an integer.

7. What is the purpose of enabling debug mode in Flask?

Debug mode in Flask is a feature that provides more detailed error messages and automatically reloads the application whenever code changes. This is extremely useful during development because it allows you to identify and fix errors quickly without needing to restart the Flask server manually. Debug mode also shows a detailed stack trace in case of an error, helping you debug issues more efficiently.

Benefits of Debug Mode:

- **Automatic code reloading:** When you modify your application code, Flask will automatically restart the server to reflect the changes.
- **Detailed error messages:** In case of an error, you get a detailed stack trace with the specific error and where it occurred in the code, making it easier to identify problems.
- **Interactive debugger:** You can even access an interactive Python debugger in the browser when an error occurs (if enabled).

8. How do you enable debug mode in a Flask application?

To enable debug mode in a Flask application, you can set the debug parameter to True when calling `app.run()`, or you can set the `FLASK_ENV` environment variable to development, which automatically enables debug mode.

Method 1: Enable debug mode using `app.run()`

```
if __name__ == '__main__':  
    app.run(debug=True)
```

This will start the Flask application in debug mode.

Method 2: Set the `FLASK_ENV` environment variable (preferred for production environments)

You can also enable debug mode by setting the `FLASK_ENV` environment variable to development.

On Windows (Command Prompt):

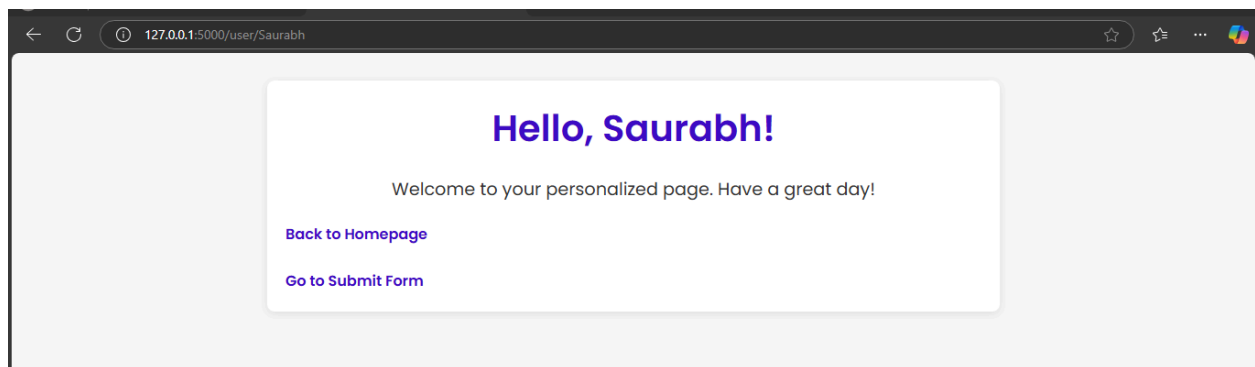
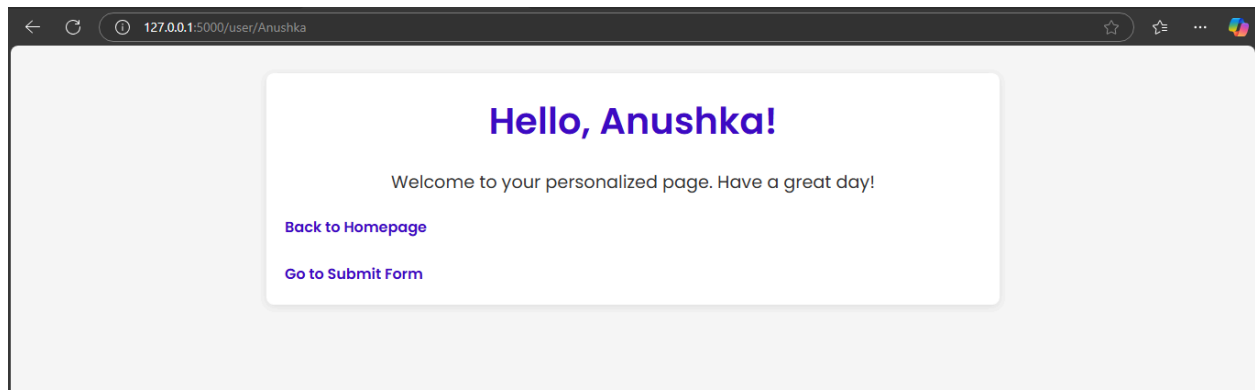
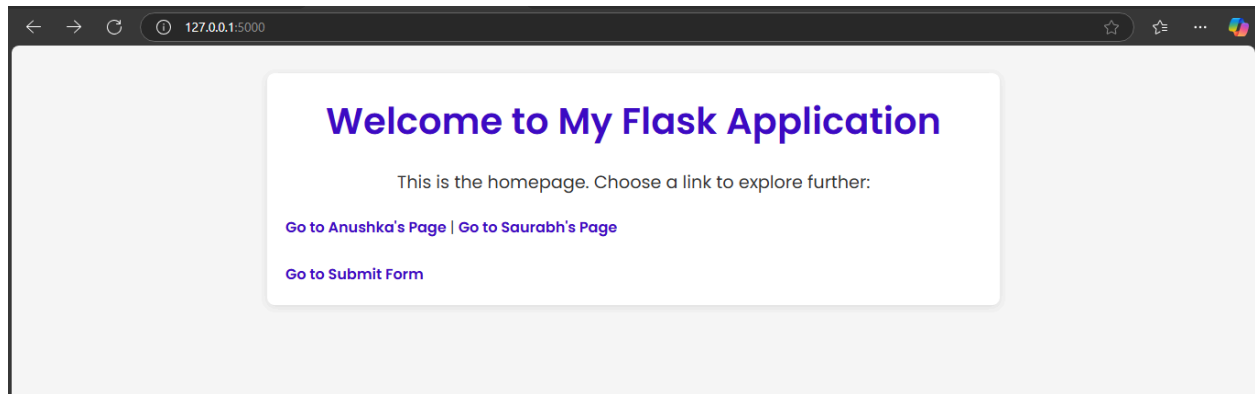
```
set FLASK_ENV=development
```

Then run your Flask app as usual:

```
python app.py
```

This will also enable debug mode and show helpful error messages in case of an issue.

OUTPUT :



127.0.0.1:5000/submit

Submit Your Details

Name:

Age:

Submit

[Back to Homepage](#)

127.0.0.1:5000/submit

Submission Successful!

Thank you for submitting your details.

Name: Anushka

Age: 21

[Back to Homepage](#)

Github link : https://github.com/Anushka3204/Exp_4_WebX_flask

Conclusion : The Flask application successfully demonstrates URL building, dynamic routes, and handling user input using GET and POST methods, providing a practical example of web development concepts.